PROGRAMMING WITH PYTHON II

Sessions 13 & 14

ie UNIVERSITY
SCHOOL OF SCIENCE & TECHNOLOGY

Index

1. The Necessity of Testing in Python
2. Unit vs. Integration Testing
3. Introduction to Pytest
4. Harnessing Fixtures for Performance
5. Testing REST API Endpoints (FastAPI)
6. Mocking and simulation

ie UNIVERSITY
SCHOOL OF SCIENCE & TECHNOLOGY

The Necessity of Testing in Python

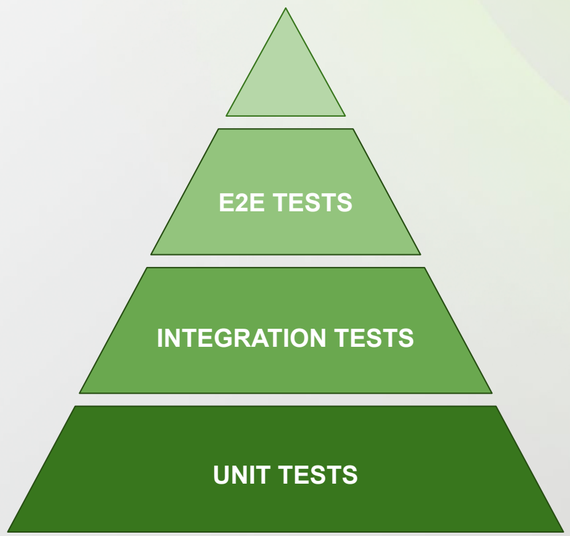
- Python is fundamentally a dynamic language; without a compile-time static type checker, you cannot be certain a function is defined until runtime.
- Tests are an insurance policy on your code: they provide confidence in correctness and make refactoring easier rather than harder.
- **The Pythonic advantage:** Python's dynamic features, while risky, make it extremely easy to write tests and diagnose malfunctioning programs.

GOAL

To make code faster, easier to read, and less prone to error.

Unit vs. Integration Testing

- **Unit tests**
  - Verify isolated functionality within a single module.
  - Often use mocks to simulate complex dependencies (like databases) to remain fast and focused.
- **Integration Tests**
  - Verify end-to-end interactions between multiple modules.
  - Essential in Python because there is no guarantee that separate modules will actually work together unless proven.



Introduction to Pytest

- Why Pytest?
  - While Unittest is the canonical standard library module, many developers find Pytest easier to use and more powerful.
- Key Features
  - Reduces boilerplate compared to TestCase subclasses.
  - High compatibility with standard Python features.\
  - Vast plugin ecosystem for specialised testing requirements.
- Configuration
  - Use a pytest.ini file at the project root to ensure tests (and doctests) are collected and executed correctly.

Introduction to Pytest

Example

```python
# --- PART 1: The Code to be Tested ---
def add_numbers(x, y):
    return x + y

# --- PART 2: The Pytest Unit Test ---
def test_addition():
    # We simply assert that the condition is True
    assert add_numbers(3, 7) == 10

def test_negative_numbers():
    assert add_numbers(-1, -1) == -2
```

```
python3 -m pip install pytest
```

Harnessing Fixtures for Performance

- A test harness (or fixture) is the environment set up before a test runs to ensure isolation.
- **Setup and Teardown**
  - **Isolation:** Each test should run in a clean environment to prevent side effects from leaking.
  - **Computational Cost:** Some setups (like starting a database) are expensive; global fixtures can run once per module to save time.
- **Encapsulating Dependencies**
  - Refactoring code to encapsulate dependencies within classes makes them significantly easier to mock and test.

Harnessing Fixtures for Performance

Example

```python
import pytest

# --- PART 1: The Complex Code (A Bank Account) ---
class BankAccount:
    def __init__(self, initial_balance=0):
        self.balance = initial_balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Deposit amount must be positive")
        self.balance += amount

    def withdraw(self, amount):
        if amount > self.balance:
            raise ValueError("Insufficient funds")
        self.balance -= amount
        return self.balance
```

ie UNIVERSITY
SCHOOL OF SCIENCE & TECHNOLOGY

```
python3 -m pip install pytest
```

Harnessing Fixtures for Performance

Example

```python
# --- PART 2: The Pytest Setup (Fixture) ---
# This runs before EVERY test function.
# It creates a fresh account with $100 so tests don't interfere with each other.
@pytest.fixture
def account():
    return BankAccount(initial_balance=100)

# --- PART 3: The Tests ---

def test_initial_balance(account):
    # The 'account' argument is automatically filled by the fixture above
    assert account.balance == 100

def test_deposit(account):
    account.deposit(50)
    assert account.balance == 150

def test_withdraw(account):
    account.withdraw(30)
    assert account.balance == 70

def test_withdraw_insufficient_funds(account):
    # We expect a ValueError if we try to withdraw $200 from a $100 account
    # This block passes ONLY if the error is raised.
    with pytest.raises(ValueError) as excinfo:
        account.withdraw(200)
```

Testing REST API Endpoints (FastAPI)

- The TestClient
  - FastAPI provides a TestClient (based on HTTPX) that makes testing endpoints extremely simple.
- Async Testing
  - Because FastAPI is built on Starlette and AnyIO, it supports native asynchronous tests.
  - To avoid blocking the event loop during tests, slow I/O operations can be delegated to thread pools.
- Validation at the Boundary
  - Tests should verify that the API "Fails Fast"—rejecting invalid data via Pydantic before it reaches business logic.

Testing REST API Endpoints (FastAPI)

Example

```python
from fastapi import FastAPI, HTTPException
from fastapi.concurrency import run_in_threadpool
import time

app = FastAPI()

# 1. A simulated blocking function (e.g., image processing, complex math)
def heavy_cpu_task(data: str):
    time.sleep(1)  # Simulates work
    if data == "crash":
        raise ValueError("Critical worker failure")
    return f"Processed {data}"

@app.get("/compute/{item_id}")
async def compute_endpoint(item_id: str):
    try:
        # 2. Offload to thread pool so we don't block the main event loop
        # The 'await' here will pause this specific request, but release the loop
        result = await run_in_threadpool(heavy_cpu_task, item_id)
        return {"result": result}

    except ValueError as e:
        # 3. FAIL FAST: If the thread errors, we catch it instantly here
        raise HTTPException(status_code=500, detail=str(e))
```

ie UNIVERSITY
SCHOOL OF SCIENCE & TECHNOLOGY

Testing REST API Endpoints (FastAPI)

Example

```python
def test_delegation_success():
    """Verify the thread pool returns data correctly."""
    response = client.get("/compute/safe_data")
    assert response.status_code == 200
    assert response.json() == {"result": "Processed safe_data"}

def test_api_fails_fast_on_thread_error():
    """
    Verify that if the background thread crashes, the API
    returns a 500 error immediately.
    """
    # We send "crash" which triggers the ValueError in our function
    response = client.get("/compute/crash")

    # The API should catch the thread's exception and return 500
    assert response.status_code == 500
    assert response.json() == {"detail": "Critical worker failure"}
```

Mocking and Simulation

- **Mocking vs Faking**
  - Mocks provide expected responses to specific calls, whereas Fakes provide simplified but working implementations (like an in-memory database).
- **The patch Mechanism**
  - The patch family of functions allows you to temporarily reassign attributes of a module or class during a test.
- **Verification**
  - It is not enough to check the return value; you must verify that the dependent functions were called with the correct parameters.
- **Nominal Typing**
  - Type hints assist in ensuring you are passing the correct "nominal" types to your functions, reducing runtime AttributeErrors.

Mocking and Simulation

- **Faking**

```python
# --- THE FAKE ---
class FakeRepository:
    def __init__(self):
        self.users = []  # Simple in-memory storage

    def save(self, user):
        self.users.append(user)  # Working logic!
```

- **Mocking**

```python
# Setup: Create a mock based on the empty class
# This mock has NO logic. It's just a spy.
mock_repo = mocker.Mock(spec=UserRepository)
```

The patch mechanism

api.py file. It uses the requests library.

```python
import requests

def get_random_fact():
    # We want to fake this call
    response = requests.get("https://uselessfacts.jsph.pl/random.json")

    # We assume the API returns JSON like: {"text": "Cats sleep 70% of their
    data = response.json()
    return data["text"]
```

mocker.patch mocks get function and mock_response mocks the result this function returns.

```python
# test_api_mocker.py
import api

def test_get_fact_with_mocker(mocker):
    # 1. Patch 'requests.get' inside the 'api' module
    mock_get = mocker.patch("api.requests.get")

    # 2. Setup the fake return value
    # We need to mock the .json() method on the response object
    mock_response = mock_get.return_value
    mock_response.json.return_value = {"text": "Mocked Fact"}

    # 3. Run the code
    result = api.get_random_fact()

    # 4. Verify
    assert result == "Mocked Fact"
    # Verify the URL was correct
    mock_get.assert_called_once_with("https://uselessfacts.jsph.pl/random.json")
```

Verification

The source code

```python
# app.py

class EmailService:
    def send_email(self, recipient, subject, body):
        # In real life, this connects to SMTP/AWS/SendGrid
        pass

class UserManager:
    def __init__(self, email_service):
        self.email_service = email_service

    def register(self, username, email):
        print(f"Saving {username} to database...")

        # --- The Interaction we want to VERIFY ---
        self.email_service.send_email(
            recipient=email,
            subject="Welcome!",
            body=f"Hello {username}, thanks for joining."
        )
```

The verification

```python
import pytest
from app import UserManager, EmailService

def test_registration_sends_email(mocker):
    # 1. SETUP: Mock the dependency
    # We create a mock version of EmailService
    mock_email_service = mocker.Mock(spec=EmailService)

    # Inject the mock into our manager
    user_manager = UserManager(mock_email_service)

    # 2. EXECUTE: Run the logic
    user_manager.register("Alice", "alice@example.com")

    # 3. VERIFY: Did the mock get called correctly?

    # EXACT Verification: Checks the exact arguments used
    mock_email_service.send_email.assert_called_once_with(
        recipient="alice@example.com",
        subject="Welcome!",
        body="Hello Alice, thanks for joining."
    )
```

Verification

Here are the most common ways to verify calls in Pytest/Mock:

Method	What it verifies
mock.assert_called()	Called at least once (arguments don't matter).
mock.assert_called_once()	Called exactly 1 time (arguments don't matter).
mock.assert_called_with(x, y)	Called at least once with exactly these arguments as the *most recent* call.
mock.assert_called_once_with(x, y)	Called exactly 1 time, with exactly these arguments.
mock.assert_any_call(x, y)	Was called with these args at some point (even if called 10 times with other args).
mock.assert_not_called()	Ensure it was NEVER called (good for error logic).

Lets practice

ie UNIVERSITY
SCHOOL OF SCIENCE & TECHNOLOGY

Bibliography

- Fluent python (Luciano Ramalho)
  - Hands-On Approach
- Effective communication (Brett Slatkin)
  - Chapters 9: Testing and Debugging
- Pytest documentation
  - https://docs.pytest.org/en/stable/

ie UNIVERSITY
SCHOOL OF SCIENCE & TECHNOLOGY